



POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Informatyki

Praca dyplomowa magisterska

TEMAT PRACY DYPLOMOWEJ

Szymon Krzysztof Gogulski, 147403

Promotor
dr inż. Michał Melosik

POZNAŃ 2026

Tutaj będzie karta pracy dyplomowej;
oryginał wstawiamy do wersji dla archiwum PP, w pozostałych kopiach wstawiamy ksero.

Spis treści

1	Wstęp	1
1.1	Zapotrzebowanie na niezawodne źródła losowości	1
1.2	Cel pracy	1
1.3	Charakterystyka literatury	1
1.4	Układ pracy	2
2	Podstawy teoretyczne	3
2.1	Entropia Shannon’a	3
2.2	PRNG a TRNG	3
2.3	Oscylator pierścieniowy	4
2.4	Architektura modułu neoTRNG	4
2.5	Korekcja źródła	5
2.5.1	Korektor Von-Neumanna	5
2.6	Metody przetwarzania końcowego	5
2.6.1	XOR	5
2.6.2	CRC	5
2.6.3	Ekstraktor Toeplitz’a	5
2.6.4	AES-256	5
2.7	Metody badania źródeł entropii	5
2.7.1	ENT	5
2.7.2	NIST SP 800-22	5
2.7.3	NIST SP 800-90B	5
2.8	Moduły testowe	5
2.8.1	RCT	5
2.8.2	APT	5
2.9	Sprzęt	5
2.9.1	Digilent CoraZ7-07S	5
2.9.2	Digilent Nexys 4 DDR	5
3	Testy bazowe	6
3.1	Modyfikacja	6
3.2	ENT	6
3.3	NIST SP 800-22	7
3.4	NIST SP 800-90B	8
3.5	Zużycie zasobów	8
4	Analiza rozmieszczenia	10

5	Faza algorytmiczna i optymalizacja ekstrakcji	13
6	Faza odporności i monitorowanie online	14
7	neoTRNG na FPGA dla RPI	15
8	Zakończenie	16
	Literatura	17
A	Kod źródłowy	18
A.1	neoTRNG_mod_1.vhd	18
B	Raport	19
C	Notatki	20

Spis rysunków

2.1	Model źródła entropii. Źródło: [5]	3
2.2	Przykład prostego oscylatora pierścieniowego.	4
2.3	Architektura neoTRNG. Źródło: [4]	4
4.1	Rozproszone komórki entropii.	11
4.2	Zgrupowane komórki entropii.	12

Spis tabel

3.1	ENT: Entropy bits per byte.	6
3.2	3 RO : 5 - 7 - 9	7
3.3	3 RO : 3 - 5 - 7	7
3.4	2 RO : 3 - 5	8
3.5	NIST SP 800-90: Non-IID entropy assessment	8
3.6	Zużycie zasobów układu.	9

Rozdział 1

Wstęp

1.1 Zapotrzebowanie na niezawodne źródła losowości

Generatory liczb losowych stanowią podstawę algorytmów współczesnej kryptografii [3]. Generowanie par kluczy kryptograficznych oraz tworzenie podpisów cyfrowych nie byłyby możliwe bez odpowiednich źródeł losowości. Jednocześnie generatory liczb losowych, które nie spełniają wymagań statystycznych oraz miar losowości, mogą znacząco osłabiać bezpieczeństwo stosowanych algorytmów. Przesłanki te stanowią motywację do podjęcia badań nad oceną dostępnych generatorów.

1.2 Cel pracy

Celem niniejszej pracy magisterskiej jest implementacja oraz analiza generatorów losowości bazujących na oscylatorach pierścieniowych. Badane moduły będą implementowane jako układy cyfrowe realizowane na płytach deweloperskich wyposażonych w układy FPGA.

Kryteria oceny generatorów obejmować będą: zestaw testów statystycznych zdefiniowanych w publikacji NIST SP 800-22 [2], zestaw testów do oceny entropii opisany w publikacji NIST SP 800-90B [5] oraz program do oceny generatorów pseudolosowych ENT [1]. Wraz z wynikami testów zestawione zostaną informacje o zajętości zasobów sprzętowych poszczególnych implementacji. Badane moduły będą stanowić modyfikacje bazowego rozwiązania neoTRNG przedstawionego w repozytorium [4]. Modyfikacje obejmą aspekty takie jak: liczba zastosowanych oscylatorów, zastosowanie korekcji źródła (np. korektor Von Neumanna), metody przetwarzania końcowego (post-processing) oraz fizyczne rozmieszczenie elementów w strukturze FPGA.

Osobna faza badań poruszy kwestie analizy odporności układu oraz mechanizmów monitorowania. Zbadany zostanie wpływ modułów testowych (Health Tests) oraz wpływ warunków środowiskowych na jakość badanego źródła entropii.

Dodatkowy etap implementacyjny obejmie porównanie zastosowania neoTRNG jako generatora entropii dla komputera jednopłytkowego Raspberry Pi z rozwiązaniem komercyjnym, takim jak moduł Quantis.

1.3 Charakterystyka literatury

Badania zostaną przeprowadzone w oparciu o rekomendacje branżowe dotyczące projektowania źródeł entropii oraz publikacje naukowe związane z analizowaną tematyką.

1.4 Układ pracy

Struktura pracy obejmuje osiem rozdziałów oraz dodatkową sekcję załączników.

Rozdział pierwszy przedstawia motywację podjęcia badań w obszarze generatorów entropii oraz określa założenia i cele pracy.

Rozdział drugi koncentruje się na podstawach teoretycznych, omówione zostaną definicje związane z tematyką pracy, architektura badanego źródła entropii, a także metody korekcji źródła, metody przetwarzania końcowego oraz techniki oceny (testy statystyczne i moduły testowe).

Rozdział trzeci rozpoczyna część badawczą pracy i obejmuje analizę wpływu korekcji źródła oraz metod przetwarzania końcowego na poziom entropii i zajętość zasobów sprzętowych.

Rozdział czwarty poświęcony jest analizie wpływu fizycznego rozmieszczenia oscylatorów w strukturze układu na poziom entropii.

Rozdział piąty przedstawia porównanie różnych metod przetwarzania końcowego.

Rozdział szósty rozszerza zakres badań o zagadnienia związane z monitorowaniem działania źródła oraz wpływem warunków środowiskowych.

Rozdział siódmy omawia dodatkowe aspekty implementacyjne oraz zawiera porównanie modułu wykorzystującego oscylatory pierścieniowe z komercyjnym generatorem Quantis.

Rozdział ósmy kończy pracę i podsumowuje najważniejsze wnioski wynikające z przeprowadzonych badań.

Rozdział 2

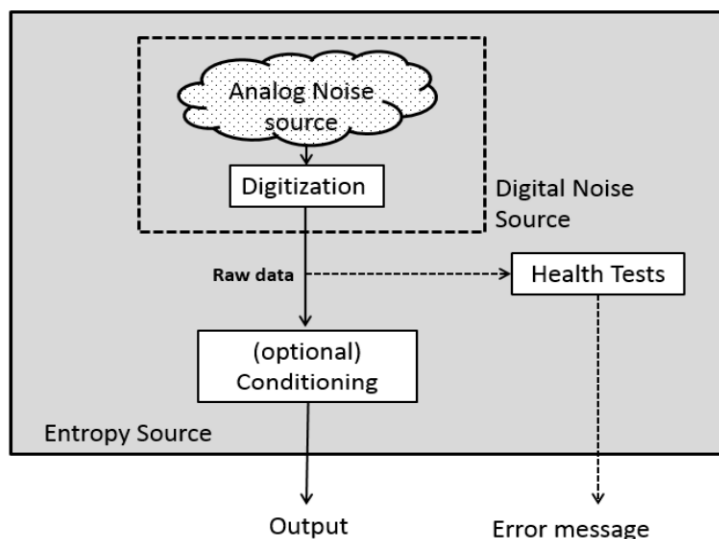
Podstawy teoretyczne

Projektowanie sprzętowych generatorów liczb losowych wiąże się z syntezą wiedzy z wielu dziedzin: elektroniki cyfrowej, fizyki, przetwarzania sygnałów, kryptografii oraz teorii informacji. W niniejszej części przedstawiono najważniejsze pojęcia związane z tematyką pracy. Wyjaśnione zostaną zarówno pojęcia fundamentalne jak losowość według Shannon'a oraz bardziej złożone zagadnienia architektury źródła entropii.

2.1 Entropia Shannon'a

1. Wstaw podręcznikową definicję, pamiętaj o \cite
2. porównaj dwa sygnały, 010101010101010101, i 11001011101110100011 za pomocą ENT

2.2 PRNG a TRNG



RYSUNEK 2.1: Model źródła entropii. Źródło: [5]

1. Omów różnicę między PRNG i TRNG, najlepiej na podstawie źródła
2. Podkreśl znaczenie pochodzenia szumu ze źródła analogowego
3. Opisz model źródła nist

2.5 Korekcja źródła

2.5.1 Korektor Von-Neumanna

2.6 Metody przetwarzania końcowego

2.6.1 XOR

2.6.2 CRC

2.6.3 Ekstraktor Toeplitz'a

2.6.4 AES-256

2.7 Metody badania źródeł entropii

2.7.1 ENT

2.7.2 NIST SP 800-22

2.7.3 NIST SP 800-90B

2.8 Moduły testowe

2.8.1 RCT

2.8.2 APT

2.9 Sprzęt

2.9.1 Digilent CoraZ7-07S

2.9.2 Digilent Nexys 4 DDR

Rozdział 3

Testy bazowe

W niniejszym rozdziale przeanalizowano wpływ zastosowania korekcji źródła oraz przetwarzania końcowego na poziom generowanej entropii w domyślnej implementacji neoTRNG. Rozważane elementy architektury zostały przedstawione na rysunku 2.3 jako moduły „de-biasing” oraz „sampling”. W implementacji bazowej zastosowano korektor von Neumanna oraz mechanizm kompresji entropii oparty na CRC. W ramach przeprowadzonych testów porównano cztery warianty konfiguracji generatora:

1. moduł z korektorem von Neumanna oraz CRC (domyślny),
2. moduł z korektorem von Neumanna bez CRC,
3. moduł bez korektora z aktywnym CRC,
4. moduł bez korektora i bez CRC.

Zbadane zostały kolejno moduły z:

1. trzema pętlami oscylatorów po 5, 7 oraz 9 inwerterów,
2. trzema pętlami oscylatorów po 3, 5 oraz 7 inwerterów,
3. dwiema pętlami oscylatorów po 3 i 5 inwerterów.

3.1 Modyfikacja

Testy przeprowadzono na zmodyfikowanym module neoTRNG. Moduł rozszerzono o dwa parametry generyczne *ENABLE_VON_NEUMANN* oraz *ENABLE_CRC* w celu łatwej konfiguracji. Zmodyfikowany plik źródłowy znajduje się w załączniku A neoTRNG_mod_1.vhd.

Każdy z przypadków testowych uruchomiono na sprzęcie, a uzyskane wyniki zapisano w odpowiednich plikach.

3.2 ENT

TABELA 3.1: ENT: Entropy bits per byte.

Konfiguracja	3RO 5-7-9	3RO 3-5-7	2RO 3-5
true/true	7.999998	7.999998	7.999997
true/false	7.928600	7.856006	7.368478
false/true	7.999988	7.999995	7.999742
false/false	7.404643	7.921076	7.617650

3.3 NIST SP 800-22

Note: The minimum pass rate for each statistical test with the exception of the random excursion (variant) test is approximately = 96 for a sample size = 100 binary sequences[2].

Note: The minimum pass rate for each statistical test with the exception of the random excursion (variant) test is approximately = 96 for a sample size = 100 binary sequences[2].

Legend: **sp** - subtests passed (zaliczone podtesty).

Legend: ***** - test niezaliczony.

Legend: **ENABLE_VON_NEUMANN / ENABLE_CRC** - wybrana konfiguracja.

TABELA 3.2: 3 RO : 5 - 7 - 9

Statistical Test	true/true		true/false		false/true		false/false	
Frequency	98/100		100/100		97/100		100/100	
BlockFrequency	99/100		100/100		98/100		100/100	
CumulativeSums	98/100		100/100		97/100		99/100	
CumulativeSums	98/100		100/100		97/100		100/100	
Runs	100/100		0/100	*	97/100		0/100	*
LongestRun	100/100		0/100	*	99/100		0/100	*
Rank	99/100		99/100		99/100		99/100	
FFT	100/100		2/100	*	98/100		0/100	*
NonOverlappingTemplate	147/148	sp	8/148	sp	148/148	sp	0/148	sp
OverlappingTemplate	100/100		0/100	*	99/100		0/100	*
Universal	97/100		0/100	*	100/100		0/100	*
ApproximateEntropy	100/100		0/100	*	100/100		0/100	*
RandomExcursions	7/8	sp	1/8	sp	8/8	sp	2/8	sp
RandomExcursionsVariant	18/18	sp	18/18	sp	18/18	sp	18/18	sp
Serial	99/100		0/100	*	99/100		0/100	*
Serial	98/100		95/100	*	99/100		0/100	*
LinearComplexity	99/100		99/100		100/100		100/100	

TABELA 3.3: 3 RO : 3 - 5 - 7

Statistical Test	true/true		true/false		false/true		false/false	
Frequency	98/100		96/100		98/100		94/100	*
BlockFrequency	100/100		0/100	*	97/100		0/100	*
CumulativeSums	98/100		93/100	*	99/100		93/100	*
CumulativeSums	99/100		93/100	*	98/100		95/100	*
Runs	100/100		0/100	*	98/100		84/100	*
LongestRun	99/100		0/100	*	99/100		77/100	*
Rank	99/100		98/100		100/100		97/100	
FFT	98/100		0/100	*	99/100		7/100	*
NonOverlappingTemplate	147/148	sp	0/148	sp	148/148	sp	11/148	sp
OverlappingTemplate	100/100		0/100	*	99/100		1/100	*
Universal	98/100		0/100	*	96/100		0/100	*
ApproximateEntropy	100/100		0/100	*	99/100		0/100	*
RandomExcursions	8/8	sp	0/8	sp	8/8	sp	4/8	sp
RandomExcursionsVariant	18/18	sp	18/18	sp	18/18	sp	18/18	sp
Serial	99/100		0/100	*	98/100		0/100	*
Serial	99/100		20/100	*	99/100		94/100	*
LinearComplexity	97/100		98/100		100/100		99/100	

TABELA 3.4: 2 RO : 3 - 5

Statistical Test	true/true		true/false		false/true		false/false	
Frequency	100/100		100/100		98/100		14/100	*
BlockFrequency	100/100		100/100		98/100		0/100	*
CumulativeSums	100/100		100/100		98/100		14/100	*
CumulativeSums	100/100		100/100		98/100		15/100	*
Runs	100/100		0/100	*	99/100		0/100	*
LongestRun	98/100		0/100	*	99/100		0/100	*
Rank	99/100		99/100		99/100		99/100	
FFT	100/100		0/100	*	98/100		0/100	*
NonOverlappingTemplate	148/148	sp	8/148	sp	148/148	sp	4/148	sp
OverlappingTemplate	99/100		0/100	*	100/100		0/100	*
Universal	98/100		0/100	*	100/100		0/100	*
ApproximateEntropy	98/100		0/100	*	97/100		0/100	*
RandomExcursions	8/8	sp	0/8	sp	8/8	sp	0/8	sp
RandomExcursionsVariant	18/18	sp	18/18	sp	18/18	sp	18/18	sp
Serial	99/100		0/100	*	98/100		0/100	*
Serial	100/100		0/100	*	99/100		0/100	*
LinearComplexity	98/100		100/100		100/100		100/100	

3.4 NIST SP 800-90B

TABELA 3.5: NIST SP 800-90: Non-IID entropy assessment

Konfiguracija	3RO 5-7-9	3RO 3-5-7	2RO 3-5
true/true	$H_{orig} = 7.918$ $H_{bit} = 0.941$	$H_{orig} =$ $H_{bit} =$	$H_{orig} =$ $H_{bit} =$
true/false	$H_{orig} = 6.653213$ $H_{bit} = 0.754510$	$H_{orig} =$ $H_{bit} =$	$H_{orig} =$ $H_{bit} =$
false/true	$H_{orig} = 7.915044$ $H_{bit} = 0.941182$	$H_{orig} =$ $H_{bit} =$	$H_{orig} =$ $H_{bit} =$
false/false	$H_{orig} =$ $H_{bit} =$	$H_{orig} =$ $H_{bit} =$	$H_{orig} =$ $H_{bit} =$

3.5 Zużycie zasobów

1	+-----+-----+-----+-----+-----+-----+-----+												
2		Site Type		Used		Fixed		Prohibited		Available		Util%	
3	+-----+-----+-----+-----+-----+-----+-----+												
4		Slice LUTs		69		0		0		14400		0.48	
5		LUT as Logic		69		0		0		14400		0.48	
6		LUT as Memory		0		0		0		6000		0.00	
7		Slice Registers		46		0		0		28800		0.16	
8		Register as Flip Flop		46		0		0		28800		0.16	
9		Register as Latch		0		0		0		28800		0.00	
10		F7 Muxes		0		0		0		8800		0.00	
11		F8 Muxes		0		0		0		4400		0.00	
12	+-----+-----+-----+-----+-----+-----+-----+												
13	* Warning! LUT value is adjusted to account for LUT combining.												

TABELA 3.6: Zużycie zasobów układu.

Konfiguracja	3RO 5-7-9	3RO 3-5-7	2RO 3-5
true/true	$LUT = 69$ $FF = 46$	$LUT = 50$ $FF = 46$	$LUT = 44$ $FF = 44$
true/false	$LUT = 65$ $FF = 39$	$LUT = 49$ $FF = 39$	$LUT = 36$ $FF = 37$
false/true	$LUT = 72$ $FF = 43$	$LUT = 61$ $FF = 43$	$LUT = 37$ $FF = 41$
false/false	$LUT = 57$ $FF = 36$	$LUT = 49$ $FF = 36$	$LUT = 35$ $FF = 34$

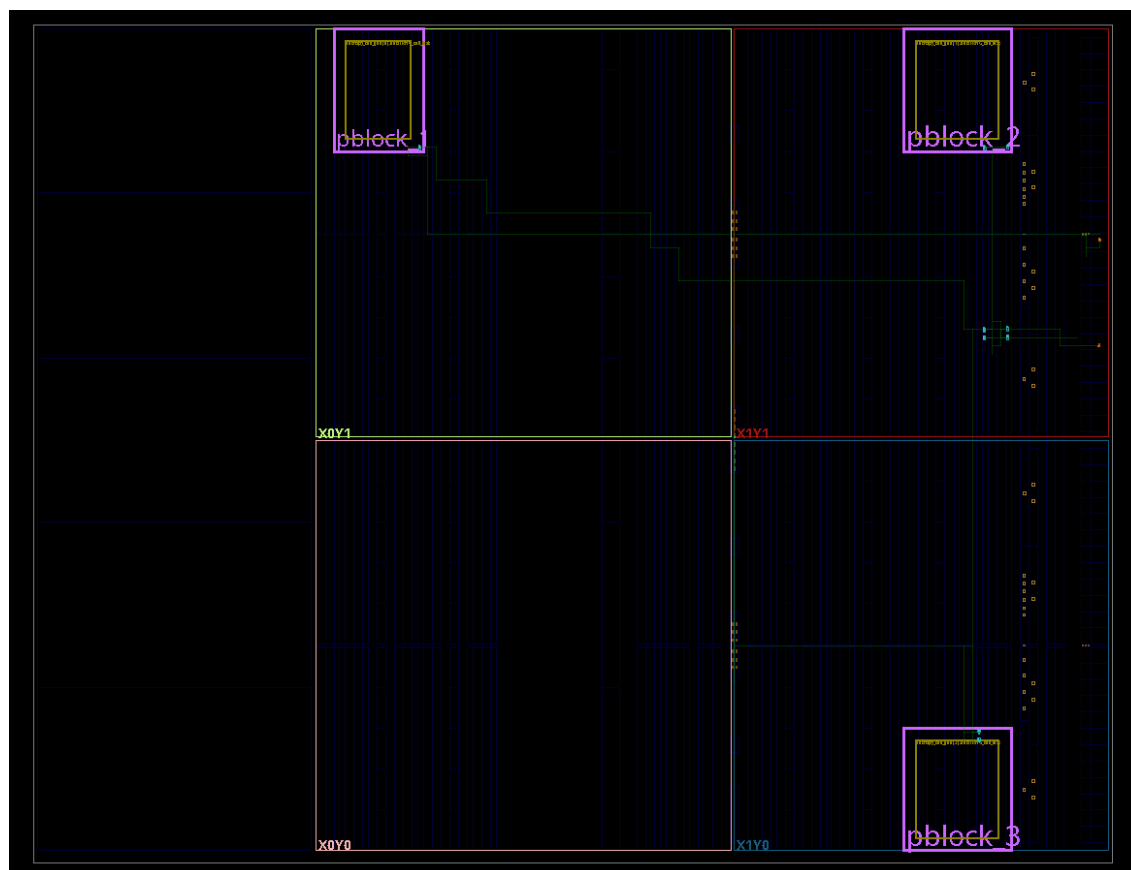
Rozdział 4

Analiza rozmieszczenia

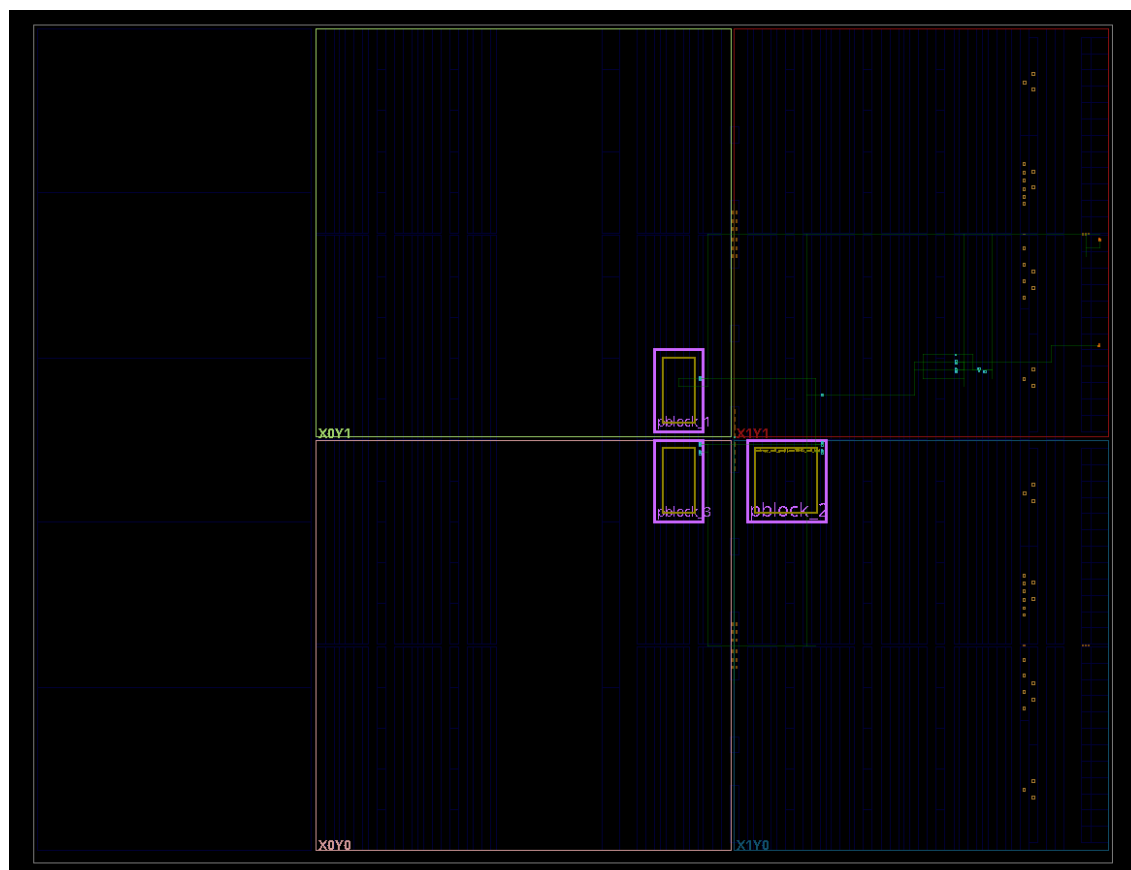
W architekturze neoTRNG zdefiniowano odrębny moduł komórki entropii. Pojedyncza komórka stanowi pojedynczą pętlę inwerterów tworzącą oscylator pierścieniowy. W ramach całego generatora możliwa jest implementacja wielu takich komórek. Każda z nich stanowi niezależne źródło entropii, a generowane sygnały są następnie łączone z wykorzystaniem operacji XOR.

Podstawą generowania entropii przez oscylator pierścieniowy jest wykorzystanie niedoskonałości wewnętrznych układu FPGA oraz wpływu czynników fizycznych, takich jak zmiany temperatury. W obrębie pojedynczego układu FPGA mogą występować lokalne różnice temperatur, które potencjalnie wpływają na poziom generowanej entropii. Regiony charakteryzujące się większym wykorzystaniem zasobów logicznych powinny osiągać wyższe temperatury niż obszary mniej obciążone.

W ramach kolejnego eksperymentu zostanie przeanalizowany wpływ fizycznego rozmieszczenia komórek entropii w strukturze układu FPGA. Rozważone zostaną przypadki, w których komórki umieszczone są bezpośrednio obok siebie, jak również konfiguracje, w których znajdują się na przeciwległych krańcach układu.



RYSUNEK 4.1: Rozproszone komórki entropii.



RYSUNEK 4.2: Zgrupowane komórki entropii.

Rozdział 5

Faza algorytmiczna i optymalizacja ekstrakcji

Domyślna implementacja generatora neoTRNG wykorzystuje ekstraktor von Neumanna oraz mechanizm kompresji entropii oparty na funkcji CRC w ramach etapu przetwarzania końcowego. W niniejszym rozdziale przeprowadzona zostanie analiza alternatywnych metod korekcji źródła oraz ekstrakcji entropii.

W ramach badań porównane zostaną różne techniki korekcji źródła, zestawione z bazowym rozwiązaniem opartym na mieszaniu entropii za pomocą operacji XOR. Analizie poddane zostaną następujące metody:

1. ekstraktor von Neumanna
2. ekstraktor Toeplitza

Pośród analizowanych metod kompresji entropii uwzględnione zostaną:

1. CRC
2. AES-256
3. SHA-256

Celem badania będzie określenie wpływu różnych metod przetwarzania końcowego na jakość statystyczną generowanej entropii oraz analiza zajętości zasobów sprzętowych poszczególnych rozwiązań.

Rozdział 6

Faza odporności i monitorowanie online

Według zaleceń dotyczących projektowania źródeł entropii wykorzystywanych do generowania bitów losowych[5], model źródła entropii można rozpatrywać jako system składający się z trzech części składowych: analogowego źródła szumu, metod przetwarzania szumu (omówionych w poprzednim rozdziale) oraz mechanizmów monitorowania i uodporniania pracy generatora, które stanowią przedmiot niniejszego rozdziału.

Omówione oraz zaprezentowanie zostaną implementacje modułów testowych:

1. RCT (Repetition Count Test) - test liczby powtórzeń,
2. APT (Adaptive Proportion Test) - adaptacyjny test proporcji.

Moduły zostaną przetestowane względem ich reakcji na zmienne warunki środowiskowe.

Rozdział 7

neoTRNG na FPGA dla RPI

Rozdział 8

Zakończenie

Literatura

- [1] Fourmilab. Ent: A pseudorandom number sequence test program.
- [2] Andrew Rukhin, Juan Soto, James Nechvatal, Miles Smid, Elaine Barker, Stefan Leigh, Mark Levenson, Mark Vangel, David Banks, N. Heckert, James Dray, San Vo, and Lawrence Bassham. Nist sp 800-22: A statistical test suite for random and pseudorandom number generators for cryptographic applications. *NIST Information Technology Laboratory*, 2010.
- [3] Lakshmi Sreekumar and P. Ramesh. Selection of an optimum entropy source design for a true random number generator. *Procedia Technology*, 2016.
- [4] stnolting. neutrng: A true random number generator for fpgas, 2016.
- [5] Meltem Sönmez Turan, Elaine Barker, John Kelsey, Kerry McKay, Mary Baish, and Michael Boyle. Nist sp 800-90b: Recommendation for the entropy sources used for random bit generation. *NIST Information Technology Laboratory*, 2018.

Dodatek A

Kod źródłowy

A.1 neoTRNG_mod_1.vhd

1

2

Dodatek B

Raport

Pytania

1. Kwestia sprzętu: trzymać się swojego czy czekać na Nexys 4?
 - Obecnie pracuję na swoim. Zależnie od decyzji dr. Kropidłowskiego pozostanę przy swoim rozwiązaniu lub powtórzę testy na Nexys.
2. Jak dodać w tekście informację o licencji neoTRNG?
3. Czy można cytować README repozytorium GitHub?
4. Czy dodać spis fragmentów kodu tak jak spis rysunków i tabel?
5. Jak powiedzieć de-biasing / sampling po polsku?
 - W tekście użyłem: de-biasing → korekcja źródła / post-processing -> przetwarzanie końcowe / sampling → próbkowanie / CRC → kompresja entropii.
 - moduł neoTRNG wykorzystuje korekcję źródła z wykorzystaniem ekstraktora von Neumanna
6. Ile MiB danych do wykonania testów? (zbierałem 10 MiB)
 - Będę zbierać po 100 MiB.
7. NIST SP 800-90B: używać testów IID czy non-IID?
 - Wykonywałem testy non-IID, ewentualnie będą do poprawy.
8. Kto jest autorem neoTRNG?
9. Jak zestawić dane z testów?
 - Wyniki testów zestawie jak w Pańskim artykule. Pozostaje kwestia rozkładów C — ustaliliśmy, że jeszcze się Pan nad tym zastanowi.
10. Publikacje naukowe do uwzględnienia w pracy – sugestie lub wymagania

Notatki

1. Najwięcej interwencji z Pańskiej strony będzie wymagał wstęp do pracy. Na razie przygotowałem wstępny zarys, jednak brakuje mu sprecyzowania najważniejszych kwestii oraz nadania tonu reszcie pracy.
2. W rozdziale teoretycznym wybrałem zagadnienia, które mogłyby być uwzględnione. Ewentualnie proszę o informację, czego brakuje, a co jest zbędne.

Dodatek C

Notatki

Do zrobienia

1. Dodaj załącznik B z kodem źródłowym
2. Przerób wykresy .png na .svg
3. Popraw schematy .svg z Vivado



© 2026 Szymon Krzysztof Gogulski

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu L^AT_EX.